

Reflection: RTS Real-Time Systems Project

(used AI to write out my thoughts and make it cleaner)

Throughout this course I've learned a lot that I wouldn't have ever even thought about learning, but I think the process taught me more through my mistakes than through my successes. The goal was to build a low-cost real-time system on FreeRTOS, with a set of periodic tasks that all had to hit their deadlines reliably. Getting there turned out to be a lot messier than the clean scheduling diagrams that I had made.

What I Would Do Differently

The biggest thing I'd change is starting measuring. For most of the project I worked from estimated execution times, I'd assume a task took "about a millisecond," assign it a priority and period based on that guess, and move on. It wasn't until near the end, when I finally learned the code to capture worst-case execution times (WCET), then I found multiple of my assumptions were off. Some tasks ran longer than I'd thought, which threw off the schedulability analysis I thought I had already finished. If I did it again, I'd set up timing instrumentation on day one and treat WCET as a measured input to the design.

I'd also save my work far more often. More than once I made a batch of changes, broke something, and then had no clean version to fall back to. Smaller, more frequent commits would have saved me hours of re-tracing my own steps.

What Was Harder Than Expected

Priority inversion was the one that really got me. On paper it's a clean concept: a low priority task holds a resource that a high priority task needs, so the high-priority task stalls waiting for it. In practice, it showed up as a task that seemed to "randomly" miss its deadline,

and it took me a long time to even realize the missed deadline wasn't random at all. Tracking it down through the shared semaphores and actually fixing it was much harder than the textbook version made it sound.

Measuring true WCET was harder than I expected, too. I assumed I could run a task a handful of times, take the longest run, and call that the worst case. But the numbers jittered from run to run depending on what else the system was doing, and getting a number I actually trusted took a lot more care than I had planned for.

More generally, the gap between “it compiles and runs” and “it meets its deadlines every single time” was wider than I appreciated going in. Getting the system to work once is easy. Getting it to behave *deterministically*, so that it holds up under worst-case conditions and not just the happy path, is the real work of a real-time system — and it's where most of my time actually went.

The Most Valuable Thing I Learned

The most valuable thing I took away is that real-time doesn't mean fast — it means predictable. Before this project I basically treated “real-time” as a synonym for “high performance,” and my instinct would have been to optimize for average speed. What I understand now is that a system that's a little slower but always meets its deadline is far more correct, for this kind of application, than a system that's usually quick but occasionally late. Determinism beats raw speed.

That shift changed how I think about design in general. I learned to turn a vague goal like “make it responsive” into concrete, verifiable numbers — periods, deadlines, worst-case execution times, and a schedulability analysis that either holds or it doesn't. Instead of just hoping something feels fast enough, I can now reason about whether it will actually meet its

timing requirements, and back that up with evidence. That habit of measuring instead of assuming is something I'll carry into every embedded or systems project I work on after this one.

Conclusion

None of this went as smoothly as I imagined at the start, but that's exactly why I got so much out of it. The project forced me past the theory and into the messy reality of timing, resource sharing, and worst-case behavior and I came out of it understanding real-time systems in a way that no lecture on its own could have taught me.